INDIVIDUAL ASSIGNMENT 2

ML PIPELINE DESIGN & MLOPS ANALYSIS

Design and Evaluation of an MLOps Pipeline for a Rule-and-AI Hybrid Automated English Speaking and Writing Scoring System

Course	DDM501 - AI in DevOps, DataOps, MLOps
Assignment	Individual Assignment 2
Weight	5%
Submission	Written Report

Operationalizing the Assignment 1 system design through repeatable experiments, workflow orchestration, quality gates, and reproducible MLOps practices.

Table of Contents

Executive Summary

1. Introduction and Assignment 1 Recap
2. ML Pipeline Design
3. Experiment Tracking and Metrics Analysis
4. Workflow Orchestration Design
5. Code Quality and Documentation
6. Reproducibility and Versioning Strategy
7. Conclusion

Executive Summary

This report extends the ML system design from Assignment 1 into a demonstrable MLOps pipeline for automated English Writing and Speaking scoring. The implementation focus is on data ingestion and validation, preprocessing, hybrid rule-and-AI scoring, experiment tracking, offline evaluation, workflow orchestration, quality gates, reproducibility, and versioning. A mini experiment runner was executed on a synthetic evaluation dataset using a mock inference provider. The results are therefore used to validate the pipeline and MLOps workflow rather than to claim real-world scoring accuracy.

Ten experiment configurations were evaluated. The baseline achieved QWK 0.400 and MAE 0.950. Enabling deterministic rules improved QWK to approximately 0.455 and reduced MAE to 0.883. Adding few-shot prompting further reduced MAE to 0.838, although QWK changed only slightly. None of the configurations passed the proposed production quality gate (QWK >= 0.75 and MAE <= 0.50), which demonstrates that the gate correctly prevents an underperforming candidate from being promoted.

1. Introduction and Assignment 1 Recap

Assignment 1 defined the production-oriented architecture for an automated English assessment system. The system targets Writing and Speaking responses, where manual grading is costly, slow, and difficult to scale. The proposed solution combines deterministic rules with AI-based semantic evaluation to provide criterion-level scores, an overall score, and formative feedback.

Assignment 2 refines that architecture into an executable pipeline design. The emphasis shifts from what the production system should contain to how data, experiments, evaluation, orchestration, deployment, and versioning should work together in a repeatable MLOps process.

1.1 Refined Scope

- Writing input: question text and candidate essay.
- Speaking input: question, audio recording, and speech-to-text transcript.
- Hybrid scoring: deterministic rule features plus AI semantic evaluation.
- Evaluation target: agreement with reference scores, operational efficiency, and safe promotion through quality gates.
- Experiment objective: demonstrate MLOps behavior using synthetic data and a mock provider, not benchmark real examiner-level accuracy.

2. ML Pipeline Design

The detailed pipeline contains all stages required for repeatable ML/MLOps experimentation: ingestion, validation, preprocessing, feature engineering, training and configuration optimization, scoring, evaluation, quality gating, registration, and deployment.

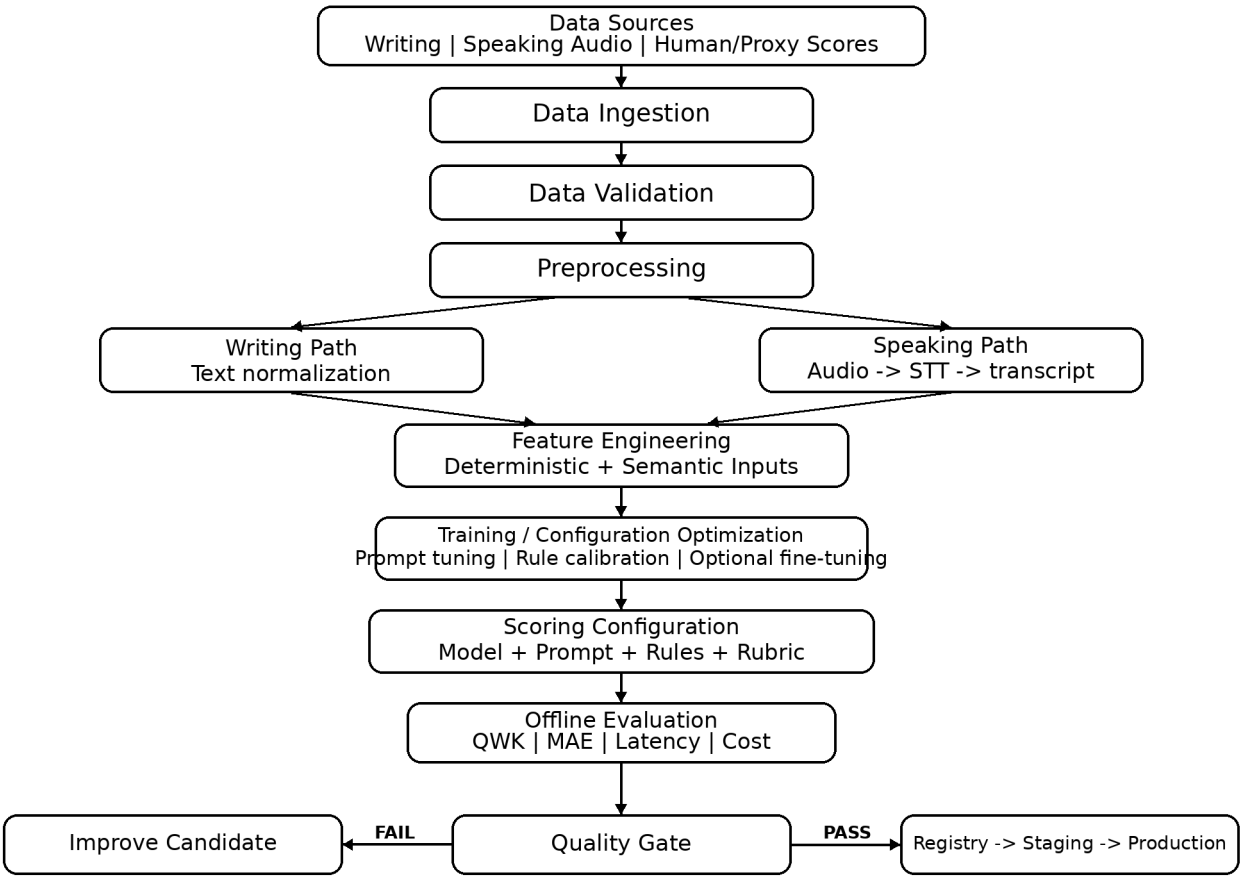


Figure 1. Detailed ML pipeline for the English scoring system.

2.1 Pipeline Stage Specifications

Stage	Input	Key operations	Output	Quality gate
Data Ingestion	Essays, audio, labels	Collect, identify, version	Raw dataset	Required fields present
Validation	Raw records	Schema, format, integrity checks	Valid records	Invalid rate below threshold
Preprocessing	Valid responses	Normalize text, check audio, generate transcript	Processed data	No empty/corrupt records
Feature Engineering	Text/transcript	Word count, duration, repetition, semantic inputs	Feature/scoring input	Features complete
Training / Optimization	Features + candidate settings	Prompt tuning, rule calibration, optional fine-tuning	Candidate scoring package	Reproducible artifact
Scoring	Features + configuration	Rule engine + AI evaluator	Structured predictions	Valid schema
Evaluation	Predictions + reference labels	QWK, MAE, latency, output validity	Evaluation report	Quality thresholds
Registry	Accepted candidate	Store version and artifacts	Registered release	Gate passed
Staging	Registered release	Deploy and smoke test	Validated candidate	Health check passed

Stage	Input	Key operations	Output	Quality gate
Production	Validated release	Serve scoring requests	Production service	Operational monitoring

2.2 Data Ingestion and Validation

The ingestion layer collects Writing submissions, Speaking transcripts or audio metadata, task prompts, and reference scores. Every record is associated with a stable sample identifier and dataset version. Validation is intentionally performed before expensive AI inference so malformed or incomplete records do not consume scoring capacity.

- Writing checks: non-empty response, valid encoding, minimum content length, required task metadata.
- Speaking checks: supported audio format, minimum duration, readable media, speech detected, prompt metadata available.
- Rejected records are logged with a reason code and excluded from the evaluation dataset.

2.3 Preprocessing and Feature Engineering

Writing preprocessing is conservative. Grammar or spelling errors must not be corrected before scoring because they are part of the assessment signal. Normalization therefore focuses on Unicode, whitespace, and metadata separation. For Speaking, the audio path adds speech detection and speech-to-text before transcript normalization.

The hybrid architecture separates deterministic features from semantic inputs. Deterministic features may include word count, sentence count, response duration, speaking rate, repetition ratio, silence ratio, and invalid-response indicators. Semantic inputs consist of the question, candidate response or transcript, rubric, task type, and selected deterministic signals.

2.4 Training and Configuration Optimization

The demonstration does not retrain the foundation model from scratch. The training stage therefore optimizes the parts controlled by the project: prompt selection, deterministic-rule calibration, score aggregation, and optional future fine-tuning. Its output is a versioned candidate scoring package that can be evaluated against the same benchmark before release.

2.5 Scoring Configuration as the Release Unit

Because the system uses a foundation model rather than training a large language model from scratch, the deployable unit is treated as a scoring configuration rather than only a model artifact. A release can include the foundation model identifier, prompt version, rule version, rubric version, preprocessing version, STT configuration, and aggregation policy. This makes prompt or rule changes auditable even when the base model does not change.

```
scoring-release-v1.3
model       = model_a
prompt      = rubric_v2
rules       = rules_v2
rubric      = assessment_v1
dataset     = synthetic_eval_v1
stt         = transcript_input
```

2.6 Quality Gates and Recovery

A candidate is evaluated before promotion. The demo quality gate uses QWK >= 0.75 and MAE <= 0.50 as model-quality targets, together with output validity and operational checks. If the candidate fails, it returns to the improvement stage; if it passes, it can be registered and tested in staging.

- Transient inference failure: retry with backoff.
- Invalid structured output: schema validation followed by retry or rejection.
- Dataset validation failure: stop the pipeline and notify the operator.
- Staging smoke-test failure: rollback or prevent production deployment.
- Operational scaling: asynchronous workers can be added behind a queue for Writing and Speaking jobs.

2.7 Design Rationale

The pipeline separates data preparation, scoring, evaluation, and deployment so each component can be tested or replaced independently. Writing and Speaking share common validation, tracking, and quality-gate components while retaining modality-specific preprocessing. This modular structure supports reuse, isolates failures, and allows workers for each modality to scale independently behind a queue.

3. Experiment Tracking and Metrics Analysis

The experiment strategy compares ten scoring configurations. Because no human-labeled production dataset was available, the pipeline uses a synthetic evaluation dataset and mock inference provider. The purpose is to validate the experiment tracking, metric computation, artifact management, and promotion logic of the MLOps pipeline.

3.1 Baseline and Experiment Variables

E01 is the baseline: model A, a basic prompt, no deterministic rules, no few-shot examples, and temperature 0.0. Later configurations introduce rubric prompts, rules, few-shot examples, alternate model identifiers, and temperature changes.

The matrix covers the required variable groups: model A/B represents the algorithm choice; temperature is the hyperparameter; rule and few-shot settings change the available scoring features; and prompt/rubric versions represent preprocessing and input-configuration alternatives.

Exp	Model	Prompt	Rules	Few-shot	Temp
E01	model_a	basic_v1	Off	No	0.0
E02	model_a	rubric_v1	Off	No	0.0
E03	model_a	rubric_v1	On	No	0.0
E04	model_a	rubric_v1	On	Yes	0.0
E05	model_a	rubric_v2	On	Yes	0.0
E06	model_a	rubric_v2	On	Yes	0.2
E07	model_b	rubric_v1	On	No	0.0

Exp	Model	Prompt	Rules	Few-shot	Temp
E08	model_b	rubric_v1	On	Yes	0.0
E09	model_b	rubric_v2	On	Yes	0.0
E10	model_b	rubric_v2	On	Yes	0.2

3.2 Metrics Strategy

The primary metrics are Quadratic Weighted Kappa (QWK) and Mean Absolute Error (MAE). QWK is appropriate for ordered scoring because a one-band error should not be treated the same as a much larger disagreement. MAE provides a direct and interpretable average distance between predicted and reference scores.

- Primary: QWK (higher is better), MAE (lower is better).
- Secondary: p50/p95 latency, invalid output rate, token counts, estimated cost per request.
- Business alignment: higher agreement can reduce human review; lower latency improves learner turnaround time; lower cost improves scalability.

3.3 MLflow Tracking Strategy

Each experiment run should log parameters, metrics, and artifacts. The runner is designed so that prompt version, rules, dataset version, model identifier, and temperature are tracked together with QWK, MAE, latency, output validity, token usage, and cost.

```
with mlflow.start_run():
    mlflow.log_params({
        "model_name": model_name,
        "prompt_version": prompt_version,
        "rule_version": rule_version,
        "dataset_version": dataset_version,
        "temperature": temperature,
    })
    mlflow.log_metrics({
        "qwk": qwk,
        "mae": mae,
        "p95_latency_ms": p95_latency_ms,
        "invalid_output_rate": invalid_output_rate,
        "cost_per_request_usd": cost_per_request_usd,
    })
    mlflow.log_artifact("artifacts/predictions.csv")
    mlflow.log_artifact("artifacts/summary.json")
```

3.4 Model Registration and Versioning

A candidate is registered only after evaluation. The registry keeps the model or scoring-package name, version, run identifier, metrics, and promotion stage so a previous release can be restored when necessary.

```
model_info = mlflow.pyfunc.log_model(
    artifact_path="scoring_package",
    python_model=scoring_model,
    registered_model_name="english-scoring-model",
)
mlflow.register_model(
    model_uri=model_info.model_uri,
    name="english-scoring-model",
)
```

3.5 Experiment Results

Exp	QWK ↑	MAE ↓	p95 ms ↓	Invalid ↓	Cost/req $	Gate
E01	0.400	0.950	0.028	0.000	0.0000	FAIL
E02	0.400	0.950	0.071	0.000	0.0000	FAIL
E03	0.455	0.883	0.080	0.000	0.0000	FAIL
E04	0.458	0.838	0.042	0.000	0.0000	FAIL
E05	0.458	0.838	0.042	0.000	0.0000	FAIL
E06	0.458	0.838	0.034	0.000	0.0000	FAIL
E07	0.455	0.883	0.097	0.000	0.0000	FAIL
E08	0.458	0.838	0.029	0.000	0.0000	FAIL
E09	0.458	0.838	0.029	0.000	0.0000	FAIL
E10	0.458	0.838	0.030	0.000	0.0000	FAIL

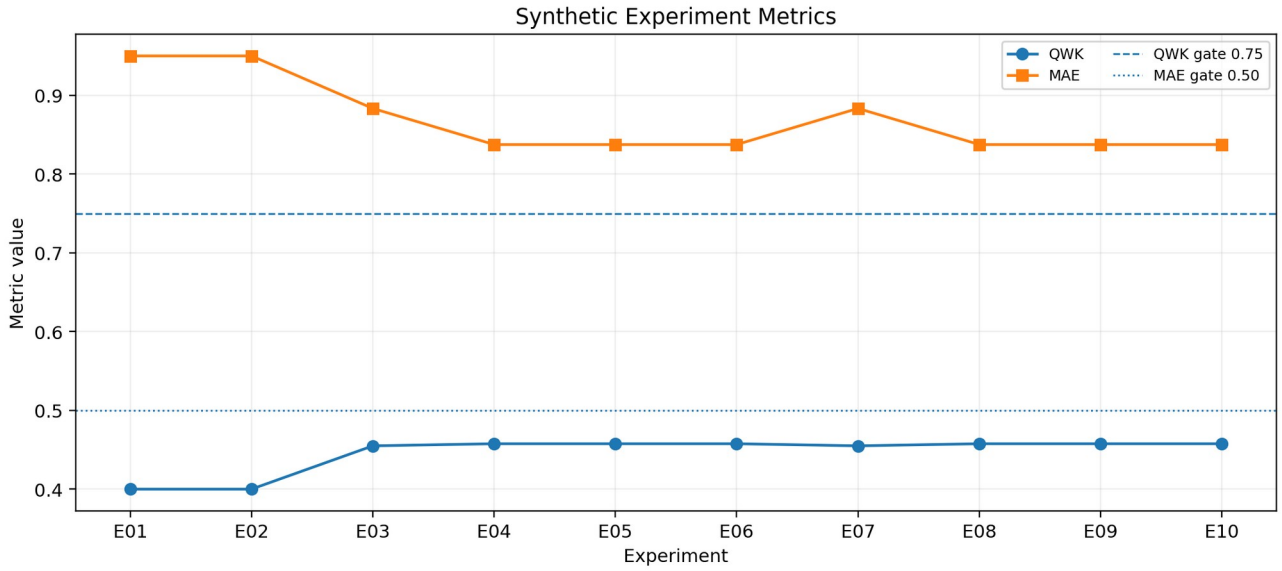


Figure 2. QWK and MAE across the ten synthetic/mock experiment configurations.

3.6 Results Analysis

The baseline configurations E01 and E02 both produced QWK 0.400 and MAE 0.950. Adding the deterministic rule layer in E03 increased QWK to approximately 0.455 and reduced MAE to 0.883. Adding few-shot examples in E04 reduced MAE further to 0.838, while QWK increased only slightly to approximately 0.458.

E04-E06 and E08-E10 produced identical quality metrics under the mock provider. This is expected in a demonstration setup because model identity, prompt version, and temperature are not connected to a real external inference service with materially different behavior. Therefore, these runs should not be interpreted as evidence that model A and model B are equivalent in production.

All ten configurations failed the proposed production quality gate. This is a valid pipeline result rather than an implementation failure: the quality gate correctly blocks underperforming

candidates from being registered or deployed. In a real deployment, the same experiment workflow would be rerun on a human-labeled evaluation dataset and real model inference.

3.7 Metric Trade-offs

E04 provides the lowest observed MAE with the simpler model-A configuration, so it offers the best balance for the demonstration. E06 does not improve quality and introduces additional response variability. Model-B configurations also provide no quality gain under the mock provider. Latency and cost values validate the logging workflow only and must not be treated as production benchmarks. In production, a small quality improvement would be accepted only when its latency and cost remain within the service targets.

3.8 Configuration Recommendation for the Demo

No configuration is recommended for production because none passed the gate. For demonstration purposes, E04 can be treated as a reference candidate because it shows the best quality observed with the simpler model-A configuration: rules enabled, few-shot enabled, rubric v1, temperature 0.0, QWK 0.458 and MAE 0.838. The purpose of selecting E04 is to demonstrate downstream tracking and staging logic, not to claim production readiness.

4. Workflow Orchestration Design

Airflow is used as the conceptual orchestration layer for the offline experiment and evaluation workflow. The DAG makes task dependencies explicit and provides retries, scheduling, failure callbacks, and operational visibility.

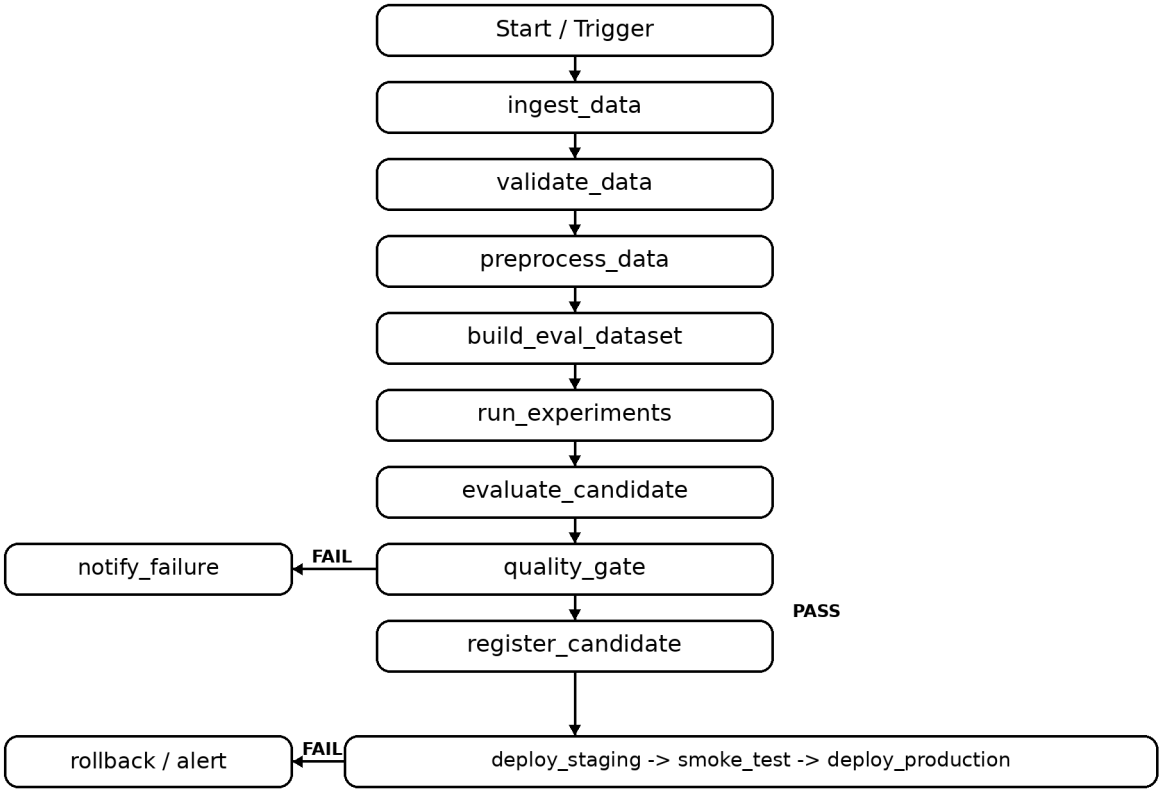


Figure 3. Airflow DAG for the experiment, evaluation, and deployment workflow.

4.1 Task Definitions and Dependencies

Task	Responsibility
ingest_data	Load versioned evaluation data
validate_data	Check schema, completeness, and integrity
preprocess_data	Normalize text/audio-derived inputs
build_eval_dataset	Create stable benchmark split
run_experiments	Execute E01-E10 configurations
evaluate_candidate	Compute QWK, MAE, latency, and validity
quality_gate	Accept or reject candidate
register_candidate	Create registered scoring release
deploy_staging	Deploy accepted candidate to staging
smoke_test	Check staging health and response schema
deploy_production	Release validated configuration
notify_failure	Alert on invalid data, task failure, or gate rejection

4.2 Scheduling and Trigger Strategy

The workflow combines scheduled evaluation with evidence-based triggers. A weekly schedule can run data-quality and benchmark checks, while a production monitoring signal can initiate

additional evaluation when quality degradation is confirmed. This avoids unnecessary retraining or prompt changes when no evidence of degradation exists.

```
@task(retries=2)
def validate_data(dataset_path: str) -> str:
    """Validate the benchmark and return its approved path."""
    return run_validation(dataset_path)

with DAG(
    dag_id="english_scoring_pipeline",
    schedule="@weekly",
    catchup=False,
) as dag:
    ingest >> validate >> preprocess >> build_eval_dataset
    build_eval_dataset >> run_experiments >> evaluate_candidate
    evaluate_candidate >> quality_gate
    quality_gate >> register_candidate >> deploy_staging
    deploy_staging >> smoke_test >> deploy_production
```

4.3 Operational Considerations

- Retries for transient API/network errors with bounded backoff.
- Task timeouts to prevent stuck experiment runs.
- Idempotent tasks so reruns do not duplicate artifacts.
- Alerts for DAG failure, repeated task failure, invalid datasets, or quality-gate rejection.
- Monitoring of task duration, failure rate, queue length, experiment count, and artifact availability.

5. Code Quality and Documentation

The implementation is organized as reusable modules instead of a single script. This makes data loading, rules, metrics, provider logic, prompt handling, and orchestration independently testable.

```
english-scoring-mlops/
├── src/
│   ├── runner.py
│   ├── provider.py
│   ├── metrics.py
│   ├── rules.py
│   ├── prompts.py
│   └── models.py
├── configs/
│   ├── base.yaml
│   └── experiments.yaml
├── prompts/
│   ├── basic_v1.txt
│   ├── rubric_v1.txt
│   └── rubric_v2.txt
├── data/
│   └── synthetic_eval.csv
├── tests/
│   └── test_metrics.py
├── Dockerfile
├── requirements.txt
└── README.md
```

5.1 Coding Standards

- PEP 8-style Python formatting and clear module boundaries.

- Type hints for public functions and data structures.
- Docstrings for experiment, metric, and provider functions.
- Comments only where implementation intent is not obvious.
- Unit tests for metric calculations and critical deterministic logic.

5.2 Configuration Loading and Environment Management

```
experiment:
  name: english-scoring-demo
  dataset_version: synthetic_eval_v1

quality_gate:
  min_qwk: 0.75
  max_mae: 0.50
  max_invalid_output_rate: 0.01

mlflow:
  experiment_name: english-scoring
```

The application loads the versioned YAML file once at startup and obtains secrets from environment variables. This keeps experiment choices reproducible without storing credentials in source control.

```
from pathlib import Path
import os
import yaml

def load_config(path: Path) -> dict:
    """Load a versioned experiment configuration."""
    with path.open(encoding="utf-8") as stream:
        return yaml.safe_load(stream)

config = load_config(Path("configs/base.yaml"))
tracking_uri = os.environ["MLFLOW_TRACKING_URI"]
```

Secrets such as API keys and service credentials are supplied through environment variables rather than hard-coded in source control. Example variables include LLM_API_KEY, MLFLOW_TRACKING_URI, DATABASE_URL, and object-storage credentials.

6. Reproducibility and Versioning Strategy

6.1 Code Versioning

Git is used to version code, prompts, configuration files, and DAG definitions. Every MLflow run should record the source commit hash so an experiment can be associated with the exact implementation that produced it. A simple main/develop/feature branch strategy is sufficient for this project.

6.1.1 Release Management

Approved releases use semantic tags such as v1.4.0 and remain linked to the source commit and MLflow run. Hotfixes branch from the production tag, while rollback restores the last registered production version.

```
git tag -a v1.4.0 -m "English scoring release"
git push origin v1.4.0
```

6.2 Data Versioning

The evaluation dataset should be immutable after release and referenced by a dataset version such as synthetic_eval_v1 or eval_v3. A production implementation could use DVC or object-storage manifests containing record count, creation timestamp, content hash, label source, and preprocessing version.

6.3 Model and Scoring-Configuration Versioning

```
scoring-release-v1.4
├── git_commit: 92f31ab
├── dataset: synthetic_eval_v1
├── model: model_a
├── prompt: rubric_v1
├── rules: enabled
├── few_shot: enabled
├── temperature: 0.0
└── container: sha256:<image-digest>
```

6.4 Random Seeds

```
import random
import numpy as np

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
```

The synthetic dataset generator uses a fixed seed so the demo dataset can be reproduced. For hosted LLMs, exact determinism may still not be guaranteed; therefore the provider/model version must also be logged.

6.5 Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src ./src
COPY configs ./configs
COPY prompts ./prompts
ENV PYTHONUNBUFFERED=1
CMD ["python", "-m", "src.runner", "--provider", "mock"]
```

6.6 Reproduction Procedure

- Checkout the recorded Git commit.
- Restore the recorded dataset version.
- Restore the experiment configuration and prompt version.
- Use the same container image and random seed.
- Run the experiment runner.
- Compare the generated metrics and artifacts with the recorded MLflow run.

7. Conclusion

This assignment operationalizes the high-level system design from Assignment 1 into a demonstrable ML/MLOps workflow. The pipeline defines explicit stages for ingestion, validation, preprocessing, training and configuration optimization, hybrid scoring, experiment tracking, offline evaluation, quality gating, orchestration, registry, deployment, monitoring, and reproducibility.

The synthetic/mock experiment demonstrates that the pipeline can compare scoring configurations, compute ordered-score metrics, log operational measures, and prevent underperforming candidates from promotion. The results also show a small improvement from the baseline when deterministic rules and few-shot prompting are introduced, while highlighting that mock inference cannot support claims about real model quality. A future production evaluation should replace synthetic proxy labels with human examiner scores and use real inference endpoints while preserving the same MLflow, Airflow, quality-gate, and versioning workflow.

Limitations of the Demonstration

- The evaluation dataset is synthetic and uses proxy labels rather than verified human examiner scores.
- The provider is a mock implementation, so latency and cost values are not representative of a real LLM service.
- Identical metrics across some experiment configurations are an artifact of the mock provider.
- The report demonstrates pipeline design and MLOps control flow; it does not claim production scoring accuracy.